



**TED UNIVERSITY**

**Faculty of Engineering**

**Department of Computer Engineering**

**CMPE472 – PROGRAMMING ASSIGNMENT 1 REPORT**

**Course Coordinator**

Emin Kuğu

**Course Assistant(s)**

Ar.Gör. Ruhi Zafer ÇAĞLAYAN

Ar.Gör. Berk Hayırlı

**Author**

Engin Samet Dede

23/03/2025

## Table Of Contents

|                                                      |          |
|------------------------------------------------------|----------|
| <b>1. Overview .....</b>                             | <b>3</b> |
| <b>2. Server Class Explanation (server.py) .....</b> | <b>3</b> |
| <b>def handle_request(client_connection): .....</b>  | <b>3</b> |
| <b>def serve_forever(): .....</b>                    | <b>7</b> |
| <b>3. Client Class Explanation (client.py) .....</b> | <b>8</b> |

## 1. Overview

In Python, the implementation of a mini weather prediction system using socket programming is carried out in this assignment. The system contains a server and a client. The weather data file `weathers.xlsx` is read from an Excel file on the server, which chooses a random city and sends the working client a guess asking for the temperature for the given city. The server provides feedback until the guess is correct within 10% of the correct number, three attempts (attempts) have passed or the client ends its session with the server using 'END' message.

## 2. Server Class Explanation (server.py)

Here in this part how the server class was implemented will be explained in detail using necessary screenshots.

### `def handle_request(client_connection):`

#### Reading City Names and Temperatures from `weather.xlsx`

How the city names and Temperatures were read will be explained better using the screenshot below:

```
try:
    weather_data = pd.read_excel("weathers.xlsx")
except Exception as e:
    error_msg = f"Failed to load Excel file given in the homework1 pdf: {e}"
    print(error_msg)
    client_connection.sendall(error_msg.encode())
    return

# Select a random row from the dataframe
row = weather_data.sample(1).iloc[0]
# The code reads "City" value from the excel file.
try:
    city = row["City"]
except KeyError:
    error_msg = "Excel error: 'City' not found."
    print(error_msg)
    client_connection.sendall(error_msg.encode())
    return

try:
    # Uses "Temp" column for temperature from the excel file.
    actual_temp = row["Temp"]
except KeyError:
    error_msg = "Excel error: 'Temp' not found."
    print(error_msg)
    client_connection.sendall(error_msg.encode())
    return
```

The first attempt to read an Excel file in csv format is done by the code `pd.read_excel(...)` and the file name is specified in the following format `weathers.xlsx`.

Any error from that (file not found, invalid file format, permission issues etc.) that happens is caught and an error message is:

- Printed to the server's console.
- The message is sent to the client using the method(`client_connection.sendall(...)`)
- It returns this message immediately after processing is stopped.

After reading file `weather_data` into data frame (`weather_data`) at one point code randomly selects 1 row using `sample(1)`.

The sample is accessed with `iloc 0` (the only row), stored in `row`. In effect the code selects one city and reads its temperature using this feature.

**The code tries to reach for the "City" column from the selected line:**

which raises a `KeyError` if the "City" column does not exist in the Excel file.

It will print and send an error message to the client.

This blocks the function and makes sure that nothing else can be executed.

**Extracting the "Temp" Value:**

Likewise, the code is trying to read the "Temp" column.

The same error handling routine is also triggered if the "Temp" column is missing.

**Evaluating User's guess with the city temperatures**

```
tolerance = abs(actual_temp) * 0.1 # 10% tolerance for the prediction of the weather is defined here.
```

Here we define tolerance rate as %10 percent by taking the predictions absolute value using `abs` method and multiplying it by 0.1.

```

# Send the first prompt to the client
prompt = f"You should guess the temperature for {city}:"
client_connection.sendall(prompt.encode())

attempts = 0
while attempts < 3:
    data = client_connection.recv(1024)
    if not data:
        break

    guess_str = data.decode().strip()
    #if the client ended the communication show this message
    if guess_str.upper() == "END":
        end_msg = "Connection terminated by client."
        client_connection.sendall(end_msg.encode())
        break

    try:
        guess = float(guess_str)
    except ValueError:
        msg_error_server = "Please enter a valid number or 'END' to quit."
        client_connection.sendall(msg_error_server.encode())
        continue

```

### Send the Prompt

The server creates a message through which the client must provide their guess regarding the temperature of a specific city.

Through the socket connection the server broadcasts this message to its associated client.

### Initialize the Attempts Counter

Through the server interface users obtain three possible attempts to predict the requested temperature value.

### Start the Loop

The server system continues to listen for client input within a loop system that stops when either the user wins or loses their guesses.

The `client_connection.recv(1024)` function retrieves data at most up to 1024 bytes from the connected client.

When the data parameter is empty the connection to the server might have been interrupted or lost so the server exits the loop.

### Check for “END” Command

The server converts received bytes into strings while also stripping away beginning and ending whitespaces.

The server processes an "END" command in any capitalization form by sending confirmation and breaking the main loop to exit the interaction.

The server sends a confirmation response to the communicating client before exiting the loop sequence.

The server exits the loop to terminate both processes with this command.

### Validate the Guess

The server system attempts to change the guess user input into a numeric floating-point format.

The ValueError will occur if the user enters letters in place of numbers.

An error message is sent to the client by the server while continuing to the rest of the loop iteration in order to ask the user to try again.

```
if abs(guess - actual_temp) <= tolerance:
    msg_IfSuccess = f"Congrats, Your guess is satisfying 10% tolerance. And the actual temperature is {actual_temp}."
    client_connection.sendall(msg_IfSuccess.encode())
    break
else:
    attempts = attempts + 1
    hint = {True: "Higher", False: "Lower"}[guess < actual_temp]

    if attempts < 3:
        client_connection.sendall(hint.encode())
    else:
        msg_IfFail = f"You failed to guess correctly in three attempts. The correct temperature is: {actual_temp}."
        client_connection.sendall(msg_IfFail.encode())
```

### Checking the Guess

If, the absolute difference of the user's guess and the actual temperature is less than 10% it returns true.

If the temperature is correct, it gives a success message to the client (break), meaning the server exit loop.

If the guess is not within the tolerance, it returns the attempts counter to the server.

### Handling Incorrect Guesses

- It depends upon a small approach to a dictionary-based approach (i.e. hint dictionary {True: "Higher", False: "Lower"}, and gives a hint that guess is higher or lower with the detection of this part. In this code, dictionary-based approach specifies a dictionary which directly utilizes a Boolean condition to the output value.
- In that case, if the user hasn't already used up all the three attempts, the server sends the user a hint, either 'Higher' or 'Lower'.
- If not, in the case of the third incorrect try, the server will send a failure message that will say what the temperature is correct.

**def serve\_forever():**

```
def serve_forever():
    """
    this method Initializes the server after that listens on port 8888, and finally handles incoming connections.
    """
    sock_listener = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock_listener.bind(("localhost", 8888))

    sock_listener.listen(5)
    print("Listening on port 8888")

    while True:
        client_connection, client_address = sock_listener.accept()
        print("Connection created with:", client_address)
        handle_request(client_connection)
        client_connection.close()
        print("Connection closed. New connection is being waited.\n")
```

- For among the TCP connections (AF\_INET, SOCK\_STREAM) we create a socket and bind it to localhost port 8888.
- The socket is put in listening mode and allows up to five pending connections.
- Accept waits forever, to have a client connected.
- The server prints out a message of the client's address when connects.
- A dedicated handler function (handle\_request) is provided to which the connection is passed, and this will process the client's data.
- Connection gets closed after the request is handled.
- In the case of when there is no socket descriptor for a socket address, the server continues to wait for the next incoming connection.

### 3. Client Class Explanation (client.py)

Here in this part how the client class was implemented will be explained in detail using necessary screenshots.

```
sock_client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    sock_client.connect(("localhost", 8888))
except Exception as exc:
    print("Connection error:", exc)
    return

init_message = sock_client.recv(1024).decode()
if not init_message:
    print("No message received from the server.")
    sock_client.close()
    return

print(init_message)
```

- The TCP (AF\_INET, SOCK\_STREAM) is used to create a new socket.
- The client attempts to connect to this server on the port 8888 at 'localhost'.
- After the connection fails, the function returns immediately with an error printed.
- The client reads 1024 bytes from the server and decodes it to a string.
- If nothing is received inside of the if loop the socket is closed and the function returns with a warning printed.
- Here the start of the interaction is indicated to the user's console if the initial message exists.

```
for _ in iter(lambda: True, False):
    user_input = input("Enter your guess or 'END' to quit the loop: ")
    sock_client.sendall(user_input.encode())

    server_reply = sock_client.recv(1024).decode()
    print("Server:", server_reply)

    if user_input.upper() == "END" or any(keyword in server_reply for keyword in ["Congrats", "The correct temperature"]):
        break

sock_client.close()
```

- Here I set up a loop to loop infinitely and accepting user input until certain condition is met to force break out.
- The user will input a guess (END to end the loop).
- On receipt of this input, the socket transmits this input over the socket to the server and encodes it.
- As the client receives no response from the server, it decodes, then prints to the console.



- The loop terminates if typing "END" (case does not matter or the server's reply contains any keyword from some predefined list, for example "Congrats " or "The correct temperature ").
- After the loop finishes, the client closed the socket letting server to close the session.